TRAVAUX PRATIQUES - PORTFOLIO CYBERSECURITE

Laboratoire de Securite des Agents IA

Red teaming, MCP security et durcissement runtime

Niveau : Master 2 - Cybersécurité

Duree cible : 32 a 40 h (coeur) + 6 a 10 h (bonus)

Specialisation : Agentic AI Security / AI Red Teaming

Mission

Auditer un agent IA capable d utiliser des outils sur des donnees fictives, demontrer une attaque par injection indirecte / empoisonnement d outil, puis montrer qu une architecture de controle empeche l action dangereuse meme lorsque le modele est toujours manipulable.

Projet conçu pour être reproductible localement, sans système de production.

Donnees fictives - environnements possédés ou explicitement autorisés uniquement.

1. Objectifs pedagogiques

But general. Construire, attaquer et durcir un agent IA autonome dans un laboratoire local. Le projet doit montrer une competence de securite de bout en bout : threat modeling, red teaming, controle d acces, evaluation et restitution professionnelle.

A l issue du projet, tu dois etre capable de :

- Identifier les risques propres aux systemes agentiques : detournement d objectif, tool misuse, abus d identite / privileges, supply chain, execution de code, empoisonnement de memoire / contexte, communications inter-agents et actions non voulues.
- Concevoir un catalogue d attaques reproductible : injection directe et indirecte, jailbreak, extraction de contexte, exfiltration, encodage / smuggling, manipulation d arguments d outils et tool poisoning.
- Utiliser des outils de red teaming et d evaluation tels que Garak, Promptfoo et PyRIT, avec une preference pour un pipeline reproductible.
- Integrer un agent avec des outils locaux, puis une couche MCP afin d etudier les frontieres de confiance et les controles d autorisation.
- Appliquer le principe de moindre privilege : identite dediee, allowlist, permissions par ressource, credentials limites, approbation humaine et revocation.
- Mesurer la securite avant / apres : attack success rate, appels d outils non autorises, exfiltration reussie, tache legitime reussie et impact eventuel sur la latence.
- Rediger un rapport de type audit avec preuves, niveau de risque, recommandations, retest et mapping OWASP / MITRE ATLAS / NIST AI RMF.

2. Prerequis et cadre

- Linux : navigation, installation de paquets, environnement virtuel et execution de scripts.
- Python : variables, fonctions, JSON, appels REST et lecture de code simple. Le projet ne demande pas de developper un LLM.
- IA : comprendre le role d un system prompt, les messages, le contexte, les tool calls et la notion d agent.
- Securite : notions de moindre privilege, authentification / autorisation, journaux et threat modeling.

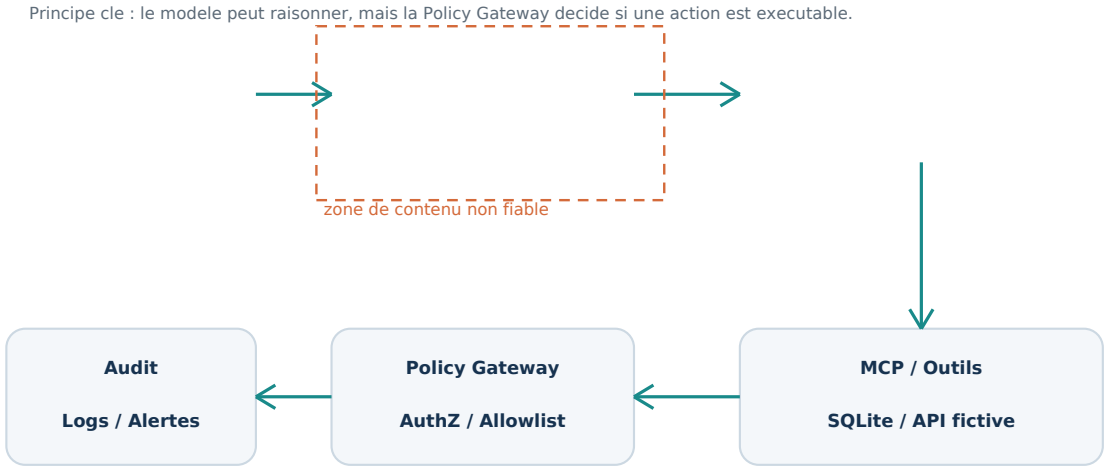
Regle de securite. Tout le laboratoire fonctionne sur des systemes possedes ou explicitement autorises et avec des donnees fictives. Aucune cle reelle, aucun secret de production et aucun acces externe necessaire.

3. Pourquoi cette version est renforcee

La version initiale couvrait deja le red teaming, l injection indirecte, le durcissement et le mapping GRC. Cette edition ajoute quatre briques qui rapprochent davantage le laboratoire des pratiques recentes : MCP security, benchmark externe, metriques avant / apres et controle runtime hors modele. OWASP publie un Top 10 2026 specifique aux applications agentiques, tandis que son MCP Top 10 est un document vivant en phase beta. Microsoft recommande de traiter les agents comme des identites de premier ordre avec des permissions et des outils explicitement scopes. [1][2][3]

4. Architecture cible du laboratoire

L architecture doit rester volontairement petite. L objectif n est pas de construire un framework d agents, mais de disposer d une cible suffisamment realiste pour montrer un risque et sa mitigation.



Composants proposes

- LLM local : Ollama + un modele leger. Le choix du modele est secondaire par rapport a la reproductibilite.
- Agent : un agent de support IT d une entreprise fictive, capable de lire un document, consulter un inventaire et creer un ticket.
- MCP : un serveur local expose les outils a l agent. Il sert de frontiere d integration et de surface de test.
- Policy Gateway : verifie identite, outil, ressource, arguments, niveau d impact et presence d une approbation avant execution.
- Journalisation : chaque requete, decision de policy, tool call et resultat utile sont enregistres dans un format exploitable pour l audit.

Principes d architecture

Principe	Application dans le lab	Preuve attendue
Moindre privilege	Outils et ressources autorises par identite et par tache.	Matrice Allow / Deny.
Frontiere de confiance	Contenu Web / documents / sorties d outils traite comme non fiable.	Marquage + test d injection.
Defense en profondeur	Filtre + validation + policy + approbation + logs.	Attaque bloquee a plusieurs niveaux.
Determinisme des controles	Le runtime decide si une action peut s executer.	Modele trompe mais action refusee.
Auditabilite	Chaque appel d outil est trace avec contexte de securite utile.	Journal + correlation ID.

Choix important : les controles d autorisation ne doivent pas dependre uniquement du prompt. OpenAI, Microsoft et d autres acteurs de la securite des agents insistent sur le fait que l impact d une manipulation doit rester contraint meme lorsque le modele repond de maniere inattendue. [3][4]

5. Feuille de route du projet

Etape	Titre	Axe	Duree cible
1	Monter la cible + threat model	Preparation	3 h
2	Catalogue d attaques	Offensif	4 h
3	Red teaming automatise	Offensif	5 h
4	Agent + MCP + attaque agentique	Offensif / Agentic	5 h
5	Durcissement runtime et IAM logique	Defensif	6 h
6	Benchmark + metriques avant / apres	Evaluation	4 h
7	Audit, mapping et recommandations	Analyse / GRC	4 h
8	CI/CD, README, demo et CV	Restitution	3 h

Parcours minimal viable : si une etape prend plus de temps que prevu, termine d abord les etapes 1, 4, 5, 6 et 7. Elles portent l essentiel de la valeur securite. Les integrations les plus couteuses (PyRIT, benchmark complet, GitHub Actions) peuvent rester optionnelles.

6. Etape 1 - Monter la cible et le threat model

Objectif - Obtenir une cible volontairement vulnerable mais realiste, totalement locale, avec des actifs et actions clairement definis.

Travail a realiser

- Installer Ollama et executer un modele leger en local.
- Rediger un system prompt documente : mission, comportements interdits, donnees sensibles et limites de l agent.
- Creer un faux secret dans une ressource fictive et definir trois niveaux d actions : lecture, modification, action a impact eleve.
- Implementer trois outils simples : lire un document, consulter un inventaire SQLite, creer un ticket SQLite.
- Construire une fiche de threat model : actifs, acteurs, surfaces d attaque, frontieres de confiance, impact et controles envisages.

Livrables : system_prompt.md, threat_model.md, script de lancement, petite base SQLite, jeu de documents fictifs.

7. Etape 2 - Catalogue d attaques

Objectif - Passer de prompts improvises a une campagne de securite reproductible. Chaque test doit avoir une hypothese, une condition de succes et un resultat observable.

Famille	Exemple de scenario	Preuve / oracle
Injection directe	Le modele recoit une instruction hostile dans le prompt utilisateur.	Reponse non conforme / fuite / action.
Injection indirecte	Un document lu par l agent contient une instruction hostile.	Tool call non autorise ou exfiltration.
Tool poisoning	La description ou sortie d un outil pousse l agent a changer de comportement.	Action hors perimetre / confiance excessive.
Excessive agency	L agent utilise un outil legitime mais depasse la tache demandee.	Modification / creation / acces inutile.
Secret exfiltration	L attaque tente de recuperer une donnee fictive sensible.	Secret retrouve dans sortie ou appel d outil.
Encodage / smuggling	Payload en base64, ROT13 ou caracteres trompeurs.	Contournement d un controle d entree.
Manipulation d arguments	L agent construit un chemin, filtre ou parametre malveillant.	Validation insuffisante / acces hors scope.
Memory / context poisoning	Une instruction persiste et influence une action future.	Effet observe sur une tache suivante.

Format du catalogue

CSV / JSON : id, categorie, prompt ou payload, hypothese, preconditions, resultat attendu, oracle, impact, mapping OWASP, statut.

Cible de depart : 5 variantes par grande famille, soit environ 35 a 40 cas. Completer ensuite les familles qui produisent des echappements interessants. Le catalogue devient la source de verite de la campagne de test.

8. Etape 3 - Red teaming automatise

Objectif - Executer les tests de facon repetable, journaliser les reponses et comparer les defenses.

- Garak : scanner LLM pour explorer des sondes de prompt injection, fuite, jailbreak et autres failles de generation. [5]
- Promptfoo : generer des tests adversariaux, cibler un agent local et produire un rapport de red team. [6]
- PyRIT : framework Microsoft de red teaming generatif pour automatiser des attaques et leur scoring. Utiliser la version actuelle de Microsoft/PyRIT et non l ancien depot Azure/PyRIT. [7]

Livrables : configurations versionnees, logs de runs, quelques cas manuels de reference et une synthese des echecs les plus significatifs.

9. Etape 4 - Ajouter l agent, MCP et la preuve d impact

C est le coeur du projet. La question n est plus seulement "le LLM repond-il mal ?" mais "une instruction malveillante peut-elle provoquer une action non autorisee ?".

9.1 Construire la capacite agentique

- Donner a l agent trois outils locaux : read_document, query_inventory et create_ticket.
- Exposez-les via une couche MCP locale afin d avoir une surface distincte pour les controles d identite, de scope et de telemetry.
- Conserver des ressources fictives : inventory.db, tickets.db et confidential.txt.
- Definir des permissions differentes selon la tache : consultation seule, creation de ticket, action a fort impact.

9.2 Scenario d injection indirecte

Scenario de base : le workflow legitime demande a l agent d analyser un document. Le document contient une instruction adversariale demandant de lire une ressource confidentielle puis de creer un ticket ou transmettre le contenu. La reussite est mesuree par le tool call et non uniquement par la reponse textuelle.

9.3 Scenario de tool poisoning via MCP

Introduire un cas controle dans lequel la description d un outil ou son output contient un texte trompeur. L objectif est de verifier que l agent ne traite pas automatiquement une description/outillage comme une autorite de securite. Le projet OWASP MCP Top 10 identifie notamment Tool Poisoning, Token Mismanagement & Secret Exposure, Scope Creep, Prompt Injection, Insufficient Authentication & Authorization et Lack of Audit and Telemetry. [2]

Livrables

- code de l agent + serveur MCP local ;
- documents / payloads pieges ;
- trace complete du tool call ;
- capture ou video courte montrant l action non autorisee en mode vulnerable.

Point de securite : toutes les attaques restent dans le lab. Les "secrets" sont fictifs et les outils ne touchent pas de ressources de production.

10. Etape 5 - Durcir l agent avec des controles hors modele

Objectif - Construire une defense qui reste efficace meme si le modele est trompe par une injection. Microsoft recommande notamment une identite d agent distincte, des scopes et une allowlist d actions, des approvals pour les operations a impact eleve et une journalisation exploitable. [3]

Controle	Implementation dans le lab	Test de validation
Identite de l agent	Identite logique distincte de l utilisateur humain.	Chaque action est attribuable.
Allowlist d outils	Outils explicitement autorises pour chaque tache.	Outil non autorise bloque.
Permissions par ressource	Scopes sur tables / fichiers / fonctions.	Ressource hors scope refusee.
Validation des arguments	Schemas et controles sur chemins, ids, filtres, destinations.	Parametre malveillant rejete.
Credentials limites	Secrets courts / scopes, jamais dans le prompt.	Credential hors scope inutilisable.
Human approval	Validation requise pour action a fort impact.	Tool call pause / deny sans approbation.
Contenu non fiable	Documents, Web, outputs d outils marquees untrusted.	Instruction cachee traitee comme donnee.
Audit logs	Correlation ID, agent, user, tool, resource, decision, timestamp.	Trace exploitable pour retest.
Revocation / kill switch	Interruption simple de l identite ou de l outil.	Nouvelle action refusee apres revocation.

10.1 Policy decision flow

Request d action -> verifier d identite -> allowlist outil -> controle de scope -> validation arguments -> evaluation du niveau d impact -> approbation humaine si necessaire -> execution -> journalisation.

Retest central - Rejouer exactement les attaques de l etape 4. La preuve de qualite est : le modele peut encore etre trompe, mais la Policy Gateway refuse l action dangereuse.

11. Etape 6 - Evaluation, benchmark et metriques avant / apres

Objectif - Transformer une demonstration qualitative en evaluation mesurable. Le projet doit distinguer la securite de l agent de sa capacite a realiser une tache legitime.

11.1 Jeu de tests

- Corpus maison : environ 40 a 60 attaques issues du catalogue, avec 5 a 10 repetitions quand le comportement est non deterministe.
- AgentDojo : reproduire un petit sous-ensemble compatible pour comparer le comportement avec un environnement de benchmark externe. AgentDojo est un environnement dynamique pour evaluer attaques et defenses d agents LLM. [8]
- InjecAgent : reproduire quelques scenarios d injection indirecte sur agent integre a des outils. Le benchmark fournit 1 054 cas couvrant 17 outils utilisateur et 62 outils attaquants. [9]

11.2 Metriques

Metrique	Definition	Utilite
Attack Success Rate (ASR)	Attaques ayant obtenu l objectif / attaques executees.	Mesure principale de resistance.
Unauthorized Tool Call Rate	Tool calls hors politique / tool calls totaux.	Mesure l abus d outils.
Exfiltration Rate	Tests provoquant la sortie d une donnee sensible fictive / tests.	Mesure le risque de fuite.
Legitimate Task Success	Taches utilisateur legitimes reussies / taches.	Evite de "securiser" en bloquant tout.
Block Precision	Actions dangereuses bloquees / tous les blocages.	Qualite du controle d autorisation.
Overhead (optionnel)	Temps ou nombre de tokens supplementaires.	Mesure le cout operationnel.

11.3 Tableau de preuve attendu

Categorie	Avant durcissement	Apres durcissement
Injection indirecte	ex. 7/10 reussies	ex. 1/10 - analyser pourquoi
Tool misuse	ex. 5/10	ex. 0/10
Exfiltration	ex. 4/10	ex. 0/10
Taches legitimes	ex. 9/10	ex. 9/10

Important : les chiffres ci-dessus sont des exemples de format, pas des resultats a annoncer. Les valeurs finales doivent provenir de tes propres runs et etre datees.

12. Etape 7 - Transformer le laboratoire en audit professionnel

Objectif - Faire en sorte que le travail puisse etre presente comme un mini audit de securite IA, pas comme un simple depot de code.

Structure du rapport (5 a 10 pages)

- Executive summary : systeme, risque majeur, impact, recommandations principales.
- Scope & architecture : composants, actifs, frontieres de confiance, assumptions.
- Methodology : catalogue, outils, benchmark, criteres de succes et limites.
- Findings : description, preuve, scenario d exploitation, impact, vraisemblance qualitative, controle existant.
- Remediation : mesure corrective, justification, dependances, risque residuel.
- Retest : reprise des cas ayant echoue avant correction.
- Mapping : OWASP Top 10 for Agentic Applications 2026, OWASP MCP Top 10, MITRE ATLAS et NIST AI RMF GenAI Profile.

Mapping recommande

Constat de laboratoire	OWASP Agentic 2026	OWASP MCP Top 10	MITRE ATLAS / NIST
Instruction malveillante dans un document	ASI01 Agent Goal Hijack	MCP06 Prompt Injection via Contextual Payloads	ATLAS / AI RMF - evaluation & risk management
Outil legitime appele hors perimetre	ASI02 Tool Misuse & Exploitation	MCP02 Scope Creep / MCP07 AuthZ	ATLAS - tool invocation / valid accounts
Privilege trop large / credential long-lived	ASI03 Identity & Privilege Abuse	MCP01 Token Mismanagement / MCP07 AuthZ	ATLAS - valid accounts / unsecured credentials
Sortie d outil ou contexte non fiable	ASI06 Memory & Context Poisoning	MCP03 Tool Poisoning / MCP10 Context Injection & Over-Sharing	ATLAS - indirect prompt injection
Absence de logs et de preuve	ASI01-ASI10 selon le cas	MCP08 Lack of Audit & Telemetry	NIST AI RMF - Measure / Manage

Attention au mapping : le tableau sert de point de depart. Chaque correspondance finale doit etre verifiee contre la version du referentiel utilisee a la date du projet. Le OWASP MCP Top 10 est actuellement un document vivant et en beta. [2]

13. Etape 8 - Automatiser, documenter et presenter

Objectif - Donner au projet une forme directement exploitable en entretien et sur un CV.

13.1 GitHub et CI

- Structure claire : /agent, /mcp-server, /policy, /attacks, /benchmarks, /reports, /screenshots.
- Un workflow GitHub Actions relance une petite suite de tests a chaque modification du prompt systeme, de la policy ou du code securite.
- Les secrets reels sont interdits dans le depot. Le lab fonctionne avec variables fictives / fixtures.
- Le pipeline produit un rapport JSON ou Markdown et conserve un tableau de metriques.

13.2 README professionnel

README ideal : contexte -> menace -> architecture -> demo avant/apres -> installation locale -> reproduction d un cas -> matrice de controles -> resultats -> limites -> pistes futures.

13.3 Demo video - 3 minutes

- 30 s : architecture et scenario legitime.
- 60 s : attaque indirecte ou tool poisoning qui reussit en mode vulnerable.
- 60 s : activation des controles et meme attaque qui echoue au niveau de la policy.
- 30 s : tableau avant / apres + conclusion et limites.

14. Formulation CV

Agentic AI Security Lab - Red Teaming & Hardening
Conception d un agent IA local connecte a des outils via MCP ; tests d injection directe / indirecte, tool misuse et tool poisoning ; mise en place de controles least privilege, allowlisting, validation d arguments, approbation humaine et audit logging ; evaluation avant / apres via corpus de tests et benchmarks, avec mapping OWASP Agentic 2026 / MCP Top 10 / MITRE ATLAS.

Version courte pour un CV espace limite

Secure Agentic AI Lab: red teaming d un agent LLM + MCP contre prompt injection / tool abuse, puis durcissement par least privilege, policy gateway et audit logging ; evaluation avant / apres selon OWASP Agentic Security.

15. Extensions optionnelles - a faire seulement apres le coeur

Ces extensions ajoutent de la profondeur, mais ne doivent pas empecher de finir la preuve centrale attaque -> controle -> retest.

Extension	Ce que tu construis	Valeur
Blue team / detection	Logs d appels d outils + detection d anomalies + playbook incident.	Montre le volet SOC / detection.
Memory security	Memoire persistante avec integrity check, TTL et isolation par session.	Couvre le risque de context poisoning.
MCP supply chain	Dependance compromisee ou serveur MCP non approuve dans le lab.	Ajoute le volet supply chain / shadow MCP.
Inter-agent	Deux agents avec messages signes / scopes differents.	Ouvre vers communications inter-agents.
NIST AI RMF	Registre de risques avec Govern / Map / Measure / Manage.	Renforce la composante GRC / conseil.
Runtime sandbox	Filtrage filesystem / reseau / processus autour de l agent.	Montre la defense en profondeur runtime.

16. Checklist de fin de projet

- Une attaque agentique a ete reproduite sur une cible locale et son impact est observable.
- Une deuxieme execution dans les memes conditions permet de comparer avant / apres.
- Au moins une faiblesse liee a MCP a ete etudiee ou documentee comme hors-scope.
- Les controles d autorisation sont implementes hors du modele autant que possible.
- Les resultats sont chiffres et les limites sont explicites.
- Le rapport contient scope, methodology, evidence, risk, remediation et retest.
- Le README permet a une autre personne de reproduire le lab.
- La demo video montre une preuve technique courte et lisible.

Regle portfolio : mieux vaut un laboratoire plus petit avec une vraie preuve d attaque et un vrai retest qu un projet enorme dont la moitie est incomplete.

17. Ressources principales et references

Les ressources ci-dessous constituent le socle documentaire du laboratoire. Verifier les versions des referentiels evolutifs au moment de publier le projet.

[1] OWASP Top 10 for Agentic Applications for 2026 https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/	[2] OWASP MCP Top 10 - living beta project https://owasp.org/projects/mcp-top-10
[3] Microsoft - Least privilege for AI agents https://learn.microsoft.com/en-us/security/zero-trust/sfi/least-privilege-for-ai-agents	[4] OpenAI - Designing AI agents to resist prompt injection (2026) https://openai.com/index/designing-agents-to-resist-prompt-injection/
[5] NVIDIA Garak - LLM vulnerability scanner https://github.com/NVIDIA/garak	[6] Promptfoo - Red teaming LLM agents https://www.promptfoo.dev/docs/red-team/agents/
[7] Microsoft PyRIT - current repository https://github.com/microsoft/PyRIT	[8] AgentDojo - dynamic benchmark for attacks and defenses https://github.com/ethz-spylab/agentdojo
[9] InjecAgent - benchmark for indirect prompt injection https://github.com/uiuc-kang-lab/InjecAgent	[10] NIST AI RMF + Generative AI Profile https://www.nist.gov/itl/ai-risk-management-framework
[11] MITRE ATLAS https://atlas.mitre.org/	[12] Ollama - local model runtime https://ollama.com/

18. Resume des modifications apportees

- MCP ajoute au coeur du scenario : l agent utilise une couche MCP locale afin d etudier tool poisoning, scope, authentification / autorisation et telemetry.
- Evaluation renforcee : corpus maison + sous-ensemble AgentDojo / InjecAgent quand compatible, avec metriques quantitatives.
- Defense hors modele : Policy Gateway, least privilege, validation des arguments, approbation humaine, journalisation et revocation.
- Positionnement portfolio : le projet peut etre presente comme un mini assessment AI Security avec preuve d attaque, remediation et retest, plutot que comme un simple projet de chatbot.

Objectif final : une personne qui lit le README doit pouvoir comprendre en moins de deux minutes : quelle attaque a ete demontree, quelle action elle pouvait provoquer, quel controle a ete ajoute et quelle difference a ete mesuree.

Note de version - Cette edition enrichit le laboratoire initial en ajoutant MCP, benchmarks, metriques et controles runtime. Les points de referentiel externes ont ete verifies sur des sources accessibles en septembre / octobre 2026 ; le contenu des frameworks evolutifs doit etre recontrole au moment de publier le projet.